단 한 번만 본다: 통합적 실시간 객체 검출 (You Only Look Once: Unified, Real-Time Object Detection)

Joseph Redmon* · Santosh Divvala*† · Ross Girshick¶ · Ali Farhadi*†
University of Washington* · Allen Institute for AI† · Facebook AI Research¶
http://pjreddie.com/yolo/

초록

본 논문은 객체 검출(object detection)에 대한 새로운 접근인 YOLO(You Only Look Once)를 제시한다. 기존 객체 검출 연구는 분류기를 재활용하여 검출을 수행한다. 이에 대해 본 연구는 객체 검출을 공간적으로 분리된 경계 상자와 이에 대응하는 클래스 확률에 대한 회귀 문제로 정식화한다. 단일 신경망이 한 번의 평가로 전체 영상으로부터 경계 상자와 클래스 확률을 직접 예측한다. 검출 파이프라인 전체가 단일 네트워크이므로 검출 성능에 대해 끝에서 끝까지 최적화할 수 있다.
통합 구조는 매우 빠르다. 기본 YOLO 모델은 초당 45프레임으로 실시간에 가깝게 영상을 처리한다. 더 작은 네트워크인 Fast YOLO는 초당 155프레임을 처리하면서도 다른 실시간 검출기 대비 mAP (mean average precision)의 두 배에 해당하는 성능을 달성한다. 최첨단 검출 시스템과 비교하면 YOLO는 국소화(localization) 오류는 더 많이 내지만 배경에 대한 거짓 양성(false positive)을 예측할 가능성은 더 낮다. 마지막으로 YOLO는 객체에 대한 매우 일반적인 표현을 학습하며, 자연 영상에서 다른 영역(예: 예술 작품)으로 일반화할 때 DPM 및 R-CNN 등 다른 검출 방법을 능가한다.

arXiv:1506.02640v5 [cs.CV] 2016년 5월 9일

1. 서론

사람은 영상을 한 번 훑어보는 것만으로도 그 안에 어떤 객체가 있는지, 어디에 있는지, 서로 어떻게 상호작용하는지 즉시 안다. 인간 시각계는 빠르고 정확하여 의식적 사고를 거의 하지 않고도 운전과 같은 복잡한 과제를 수행할 수 있다. 빠르고 정확한 객체 검출 알고리즘이 있으면 특수 센서 없이도 자동차가 스스로 주행할 수 있고, 보조 기기가 실시간 장면 정보를 사용자에게 전달할 수 있으며, 범용적이고 반응이 빠른 로봇 시스템의 가능성을 열 수 있다.

현재의 검출 시스템은 검출을 위해 분류기를 재활용한다. 객체를 검출하려면 해당 객체용 분류기를 가져와 테스트 영상의 여러 위치와 스케일에서 평가한다. 변형 가능 부품 모델(deformable parts models, DPM)과 같은 시스템은 슬라이딩 윈도우 방식으로 전체 영상의 일정 간격 위치마다 분류기를 실행한다 [10].

보다 최근의 R-CNN 등 접근법은 영역 제안(region proposal)을 사용하여 잠재적 경계 상자를 먼저 생성한 뒤, 제안된 상자들에 대해 분류기를 실행한다. 분류 후에는 경계 상자를 정제하고, 중복 검출을 제거하며, 장면 속 다른 객체에 따라 상자 점수를 다시 매기는 등의 후처리가 따른다 [13]. 이러한 복잡한 파이프라인은 느리고, 구성 요소마다 따로 학습해야 하므로 최적화하기 어렵다.

본 연구는 객체 검출을 영소(pixel)에서 경계 상자 좌표와 클래스 확률까지 이어지는 단일 회귀 문제로 재정식화한다. 이 시스템을 사용하면 영상에서 어떤 객체가 어디에 있는지 예측하기 위해 단 한 번만 본다(you only look once, YOLO).

YOLO는 상쾌할 정도로 단순하다: 그림 1을 보라. 단일 합성곱 신경망이 여러 경계 상자와 그에 대한 클래스 확률을 동시에 예측한다. YOLO는 전체 영상으로 학습하고 검출 성능을 직접 최적화한다. 이러한 통합 모델은 전통적인 객체 검출 방법에 비해 여러 이점이 있다.

첫째, YOLO는 매우 빠르다. 검출을 회귀 문제로 정식화했기 때문에 복잡한 파이프라인이 필요 없다. 테스트 시에는 새 영상에 신경망을 한 번 실행하여 검출을 예측하면 된다. 기본 네트워크는 Titan X GPU에서 배치 처리 없이 초당 45프레임을 돌리며, 빠른 버전은 초당 150프레임 이상이다. 이는 25밀리초 미만의 지연으로 스트리밍 영상을 실시간 처리할 수 있음을 의미한다. 또한 YOLO는 다른 실시간 시스템 대비 평균 정밀도의 두 배 이상을 달성한다. 웹캠에서 실시간으로 동작하는 시연은 프로젝트 웹페이지 http://pjreddie.com/yolo/에서 볼 수 있다.

둘째, YOLO는 예측할 때 영상 전체에 대해 전역적으로 추론한다. 슬라이딩 윈도우와 영역 제안 기반 기법과 달리, YOLO는 학습과 테스트 시 전체 영상을 보므로 클래스의 문맥과 겉모습에 대한 정보를 암묵적으로 부호화한다. Fast R-CNN과 같은 상위 검출 방법 [14]은 더 큰 문맥을 보지 못해 배경 패치를 객체로 오인하기도 한다. YOLO는 Fast R-CNN에 비해 배경 오류 수가 절반 미만이다.

셋째, YOLO는 객체의 일반화 가능한 표현을 학습한다. 자연 영상으로 학습하고 예술 작품으로 테스트하면, YOLO는 DPM이나 R-CNN 같은 상위 검출 방법보다 훨씬 큰 폭으로 우수하다. YOLO는 일반화가 매우 잘 되어 새로운 영역이나 예상치 못한 입력에 적용될 때도 성능 붕괴 가능성이 낮다.

YOLO는 여전히 정확도 면에서 최첨단 검출 시스템에 미치지 못한다. 영상에서 객체를 빠르게 식별할 수는 있으나, 특히 작은 객체의 정확한 국소화에는 어려움이 있다. 이러한 절충은 실험에서 추가로 살펴본다.

모든 학습·테스트 코드는 공개되어 있으며, 다양한 사전 학습 모델도 내려받을 수 있다.

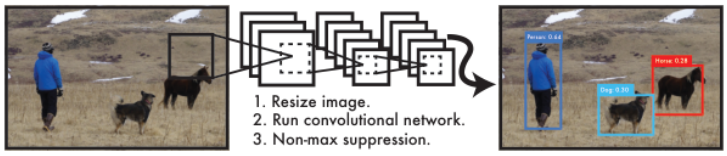


그림 1: YOLO 검출 시스템. YOLO로 영상을 처리하는 과정은 단순하고 직관적이다. 본 시스템은 (1) 입력 영상을 448×448로 크기 조정하고, (2) 영상에 단일 합성곱 네트워크를 적용하며, (3) 모델의 신뢰도에 따라 결과 검출을 임계값으로 걸러 낸다.

2. 통합 검출

객체 검출의 분리된 구성 요소를 단일 신경망으로 통합한다. 네트워크는 영상 전체의 특징을 사용하여 각 경계 상자를 예측하며, 한 영상에 대해 모든 클래스에 걸친 모든 경계 상자를 동시에 예측한다. 따라서 네트워크는 전체 영상과 그 안의 모든 객체에 대해 전역적으로 추론한다. YOLO 설계는 평균 정밀도를 유지하면서 끝에서 끝까지 학습과 실시간 속도를 가능하게 한다.

입력 영상을 $\mathrm{S} \times \mathrm{S}$ 격자로 나눈다. 객체의 중심이 어떤 격자 셀에 들어가면, 그 격자 셀이 해당 객체를 검출할 책임을 진다.

각 격자 셀은 B개의 경계 상자와 그에 대한 신뢰도 점수를 예측한다. 신뢰도는 상자가 객체를 포함할 가능성과 예측 상자의 정확도를 함께 반영한다. 형식적으로 신뢰도를 $\mathrm{Pr}(\mathrm{Object}) \cdot \mathrm{IOU}_{\mathrm{pred}}^{\mathrm{truth}}$로 정의한다. 해당 셀에 객체가 없으면 신뢰도는 0이어야 하고, 그렇지 않으면 예측 상자와 정답(ground truth) 상자 사이의 교집합 대 합집합(IOU)과 같아야 한다.

각 경계 상자는 5가지, 즉 $\mathrm{x}, \mathrm{y}, \mathrm{w}, \mathrm{h}$와 신뢰도를 예측한다. (x, y) 좌표는 격자 셀 경계에 대한 상자 중심을 나타낸다. 너비와 높이는 전체 영상에 대해 상대적으로 예측한다. 마지막으로 신뢰도 예측은 예측 상자와 정답 상자 사이의 IOU를 나타낸다.

각 격자 셀은 또한 C개의 조건부 클래스 확률 $\Pr(\text{Class}_i \mid \text{Object})$를 예측한다. 이 확률은 해당 격자 셀에 객체가 있을 때를 조건으로 한다. 상자 개수 B와 무관하게 격자 셀마다 클래스 확률 집합은 하나만 예측한다.

테스트 시에는 조건부 클래스 확률과 각 상자의 신뢰도 예측을 곱한다.

$$\Pr(\text{Class}_i \mid \text{Object}) \cdot \Pr(\text{Object}) \cdot \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) \cdot \text{IOU}_{\text{pred}}^{\text{truth}} \quad (1)$$

이 식은 각 상자에 대해 클래스별 신뢰도 점수를 준다. 이 점수는 해당 클래스가 상자 안에 나타날 확률과 예측 상자가 객체에 얼마나 잘 맞는지를 함께 부호화한다.

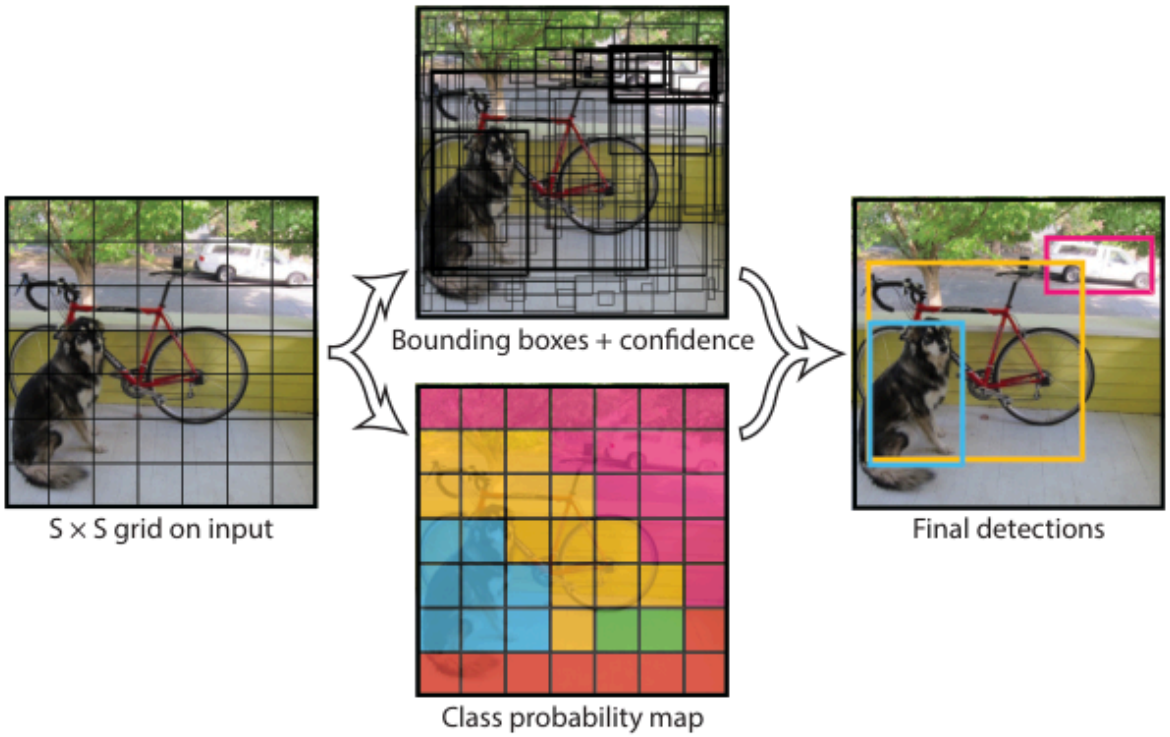


그림 2: 모델. 본 시스템은 검출을 회귀 문제로 모델링한다. 영상을 $S \times S$ 격자로 나누고, 각 격자 셀마다 B개의 경계 상자, 그 상자들에 대한 신뢰도, C개의 클래스 확률을 예측한다. 이러한 예측은 $S \times S \times (B \cdot 5 + C)$ 텐서로 부호화된다.

PASCAL VOC에서 YOLO를 평가할 때는 $S = 7$, $B = 2$를 사용한다. PASCAL VOC에는 20개의 레이블 클래스가 있으므로 $C = 20$이다. 최종 예측은 $7 \times 7 \times 30$ 텐서이다.

2.1. 네트워크 설계

이 모델을 합성곱 신경망으로 구현하고 PASCAL VOC 검출 데이터셋 [9]에서 평가한다. 네트워크의 초기 합성곱 층은 영상에서 특징을 추출하고, 완전 연결 층은 출력 확률과 좌표를 예측한다.

네트워크 구조는 영상 분류를 위한 GoogLeNet 모델 [34]에서 영감을 얻었다. 네트워크는 합성곱 층 24개와 그 뒤를 이은 완전 연결 층 2개로 이루어진다. GoogLeNet의 인셉션(inception) 모듈 대신, Lin 등 [22]과 같이 1×1 축소 층 뒤에 3×3 합성곱 층을 사용한다. 전체 네트워크는 그림 3에 나타내었다.

또한 빠른 객체 검출의 한계를 밀어 올리기 위해 설계한 YOLO의 빠른 버전인 Fast YOLO도 학습한다. Fast YOLO는 합성곱 층 수(24개 대신 9개)와 각 층의 필터 수가 더 적은 신경망을 사용한다. 네트워크 크기를 제외하면 YOLO와 Fast YOLO의 학습·테스트 설정은 동일하다.

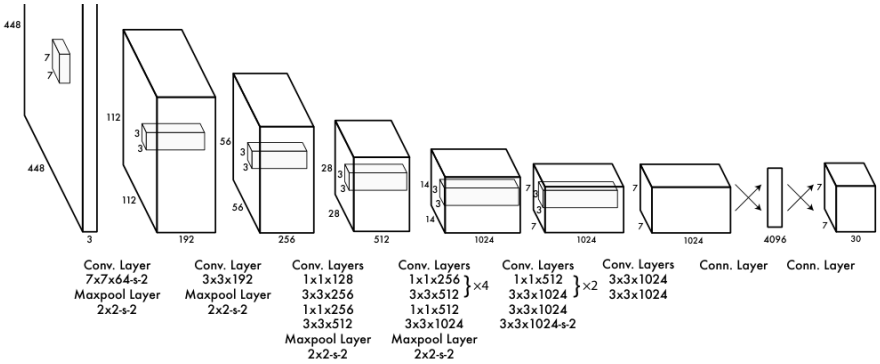


그림 3: 구조. 검출 네트워크는 합성곱 층 24개와 완전 연결 층 2개로 구성된다. 교대로 배치된 1×1 합성곱 층이 앞 층의 특징 공간을 축소한다. 합성곱 층은 ImageNet 분류 과제에서 절반 해상도 (224×224 입력)로 사전 학습한 뒤, 검출을 위해 해상도를 두 배로 올린다.

네트워크의 최종 출력은 예측의 $7 \times 7 \times 30$ 텐서이다.

2.2. 학습

합성곱 층은 ImageNet 1000클래스 경쟁 데이터셋 [30]에서 사전 학습한다. 사전 학습에는 그림 3의 처음 20개 합성곱 층 뒤에 평균 풀링 층과 완전 연결 층을 둔다. 이 네트워크는 약 일 주일 학습하여 ImageNet 2012 검증 집합에서 단일 크롭 top-5 정확도 88%를 달성하며, Caffe Model Zoo의 GoogLeNet 모델 [24]과 비슷한 수준이다. 모든 학습과 추론에는 Darknet 프레임워크를 사용한다 [26].

이후 모델을 검출용으로 변환한다. Ren 등 [29]은 사전 학습된 네트워크에 합성곱 층과 연결 층을 모두 추가하면 성능이 좋아질 수 있음을 보였다. 이에 따라 무작위 초기화된 가중치를 가진 합성곱 층 네 개와 완전 연결 층 두 개를 추가한다. 검출은 세밀한 시각 정보가 필요한 경우가 많아 네트워크의 입력 해상도를 224×224에서 448×448로 높인다.

마지막 층은 클래스 확률과 경계 상자 좌표를 동시에 예측한다. 경계 상자의 너비와 높이는 영상 너비·높이로 나누어 정규화하여 0과 1 사이에 두고, x와 y 좌표는 특정 격자 셀 위치에 대한 오프셋으로 매개변수화하여 역시 0과 1 사이로 제한한다.

마지막 층에는 선형 활성화를 쓰고, 그 외 모든 층에는 다음 누수 정류 선형 활성화(leaky ReLU)를 사용한다.

$$\varphi(\mathrm{x}) = \begin{cases} \mathrm{x} & \text{if } \mathrm{x} > 0 \\ 0.1\ \mathrm{x} & \text{otherwise} \end{cases}$$

(2)

모델 출력의 제곱합 오차(sum-squared error)를 최적화한다. 제곱합 오차는 최적화하기 쉽지만 평균 정밀도를 최대화하려는 목표와 완전히 일치하지는 않는다. 국소화 오류와 분류 오류를 동일 가중으로 두는데, 이는 바람직하지 않을 수 있다. 또한 많은 격자 셀에는 객체가 없어 해당 셀의 「신뢰도」 점수를 0 쪽으로 밀어 올려, 객체가 있는 셀에서 오는 기울기를 압도하기도 한다. 이는 학습 초기에 모델 불안정을 일으켜 발산으로 이어질 수 있다.

이를 완화하기 위해 경계 상자 좌표 예측에서 오는 손실은 키우고, 객체가 없는 상자의 신뢰도 예측에서 오는 손실은 줄인다. 이를 위해 λ_{coord}와 λ_{noobj} 두 매개변수를 사용하며, $\lambda_{\text{coord}} = 5$, $\lambda_{\text{noobj}} = 0.5$로 둔다.

제곱합 오류는 큰 상자와 작은 상자의 오류를 동일하게 가중한다. 오류 측정은 큰 상자에서의 작은 편차는 덜 중요하고 작은 상자에서는 더 중요해야 한다. 이를 부분적으로 보완하기 위해 경계 상자의 너비와 높이 자체가 아니라 그 제곱근을 예측한다.

YOLO는 격자 셀마다 여러 경계 상자를 예측한다. 학습 시에는 객체마다 책임질 경계 상자 예측기는 하나만 두고 싶다. 어떤 예측기가 정답과 현재 IOU가 가장 높은지에 따라 해당 객체를 예측할 예측기를 하나 지정한다. 이로 인해 경계 상자 예측기 사이에 전문화가 생기며, 각 예측기는 특정 크기·종횡비·클래스에 더 잘 맞추어 전체 재현율을 높인다.

학습 중에는 다음과 같은 여러 부분으로 이루어진 손실 함수를 최적화한다.

loss function:

$$
\begin{aligned}
&\lambda_{\textbf{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
&+ \lambda_{\textbf{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
&+ \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{obj}} \left(C_i - \hat{C}_i \right)^2 \\
&+ \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{\text{noobj}} \left(C_i - \hat{C}_i \right)^2 \\
&+ \sum_{i=0}^{S^2} \mathbb{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} \left(p_i(c) - \hat{p}_i(c) \right)^2 \qquad (3)
\end{aligned}
$$

학습에 사용하는 다항 손실 함수(원문 식 (3)).

여기서 $1_{\text{i}}^{\text{obj}}$는 격자 셀 i에 객체가 나타나는지, $1_{\text{i j}}^{\text{obj}}$는 격자 셀 i의 j번째 경계 상자 예측기가 그 예측에 「책임」이 있는지를 나타낸다.

손실 함수는 (앞서 논의한 조건부 클래스 확률처럼) 해당 격자 셀에 객체가 있을 때만 분류 오류에 벌점을 준다. 또한 (해당 격자 셀의 어떤 예측기보다) 정답 상자와 IOU가 가장 높을 때만 경계 상자 좌표 오류에 벌점을 준다.

네트워크는 PASCAL VOC 2007과 2012의 학습·검증 데이터로 약 135 에폭 학습한다. 2012에서 테스트할 때는 학습에 VOC 2007 테스트 데이터도 포함한다. 학습 내내 배치 크기 64, 모멘텀 0.9, 감쇠(decay) 0.0005를 사용한다.

학습률 일정은 다음과 같다. 첫 몇 에폭 동안 학습률을 10^{-3}에서 10^{-2}로 천천히 올린다. 처음부터 높은 학습률을 쓰면 기울기가 불안정해 모델이 자주 발산한다. 이후 10^{-2}로 75 에폭, 10^{-3}으로 30 에폭, 마지막으로 10^{-4}로 30 에폭 학습한다.

과적합을 막기 위해 드롭아웃과 광범위한 데이터 증강을 사용한다. 첫 번째 연결 층 뒤에 비율 0.5의 드롭아웃 층을 두어 층 간 공동 적응(co-adaptation)을 줄인다 [18]. 데이터 증강으로는 원본 영상 크기의 최대 20%까지 무작위 스케일링과 이동을 도입하고, HSV 색 공간에서 최대 1.5배까지 노출과 채도를 무작위로 조정한다.

2.3. 추론

학습과 마찬가지로 테스트 영상에 대한 검출 예측도 네트워크 평가 한 번이면 된다. PASCAL VOC에서는 영상당 98개의 경계 상자와 각 상자에 대한 클래스 확률을 예측한다. YOLO는 분류기 기반 방법과 달리 네트워크 평가가 한 번뿐이어서 테스트 시 매우 빠르다.

격자 설계는 경계 상자 예측에 공간적 다양성을 강제한다. 대개 객체가 들어갈 격자 셀은 분명하고, 네트워크는 객체마다 상자를 하나만 예측한다. 다만 여러 셀에 걸친 큰 객체나 경계 근처 객체는 여러 셀이 잘 국소화할 수 있다. 이러한 다중 검출은 비최대 억제(non-maximal suppression)로 정리할 수 있다. R-CNN이나 DPM에서처럼 성능에 결정적이지는 않지만, 비최대 억제는 mAP를 2–3 %포인트 정도 높인다.

2.4. YOLO의 한계

YOLO는 격자 셀마다 상자를 두 개만 예측하고 한 클래스만 가질 수 있게 하는 등 경계 상자 예측에 강한 공간적 제약을 둔다. 이러한 공간적 제약으로 모델이 예측할 수 있는 근접 객체 수가 제한된다. 새 떼처럼 작은 객체가 무리로 나타나는 경우에 모델은 어려움을 겪는다.

모델은 데이터로부터 경계 상자를 예측하도록 학습하므로 새롭거나 드문 종횡비·배치의 객체에는 일반화가 어렵다. 또한 입력 영상에서 여러 번 다운샘플링 층을 거치므로 상대적으로 거친 특징만으로 경계 상자를 예측한다.

마지막으로, 검출 성능을 근사하는 손실 함수로 학습하지만, 손실은 작은 경계 상자와 큰 경계 상자의 오류를 동일하게 다룬다. 큰 상자에서의 작은 오류는 대개 문제가 되지 않지만 작은 상자에서는 IOU에 훨씬 큰 영향을 준다. 주요 오류 원인은 잘못된 국소화이다.

3. 다른 검출 시스템과의 비교

객체 검출은 컴퓨터 비전의 핵심 문제이다. 검출 파이프라인은 대개 입력 영상에서 견고한 특징 집합을 추출하는 것으로 시작한다(Haar [25], SIFT [23], HOG [4], 합성곱 특징 [6]). 그다음 특징 공간에서 객체를 식별하기 위해 분류기 [36, 21, 13, 10]나 국소화기(localizer) [1, 32]를 사용한다. 이러한 분류기나 국소화기는 전체 영상에 슬라이딩 윈도우를 적용하거나 영상의 일부 영역에만 실행된다 [35, 15, 39].

YOLO 검출 시스템을 여러 상위 검출 프레임워크와 비교하며, 핵심적인 유사점과 차이를 강조한다.

가변 부품 모델. 가변 부품 모델(DPM)은 객체 검출에 슬라이딩 윈도우 접근을 사용한다 [10]. DPM은 정적 특징 추출, 영역 분류, 고득점 영역에 대한 경계 상자 예측 등 서로 분리된 파이프라인을 사용한다. 본 시스템은 이들 여러 부분을 단일 합성곱 신경망으로 대체한다. 네트워크는 특징 추출, 경계 상자 예측, 비최대 억제, 문맥 추론을 동시에 수행한다. 정적 특징 대신 네트워크가 특징을

인라인으로 학습하고 검출 과제에 맞게 최적화한다. 통합 구조는 DPM보다 빠르고 정확한 모델로 이어진다.

R-CNN. R-CNN과 그 변형은 슬라이딩 윈도우 대신 영역 제안으로 영상 속 객체를 찾는다. 선택적 탐색(Selective Search) [35]이 잠재적 경계 상자를 생성하고, 합성곱 네트워크가 특징을 추출하며, SVM이 상자를 점수화하고, 선형 모델이 경계 상자를 조정하며, 비최대 억제가 중복 검출을 제거한다. 이 복잡한 파이프라인의 각 단계를 독립적으로 정밀하게 맞춰야 하며, 결과 시스템은 테스트 시 영상당 40초 이상 걸릴 정도로 매우 느리다 [14].

YOLO는 R-CNN과 몇 가지 유사점을 공유한다. 각 격자 셀이 잠재적 경계 상자를 제안하고 합성곱 특징으로 상자를 점수화한다. 그러나 본 시스템은 격자 셀 제안에 공간적 제약을 두어 동일 객체에 대한 다중 검출을 완화한다. 또한 선택적 탐색에서 나오는 약 2000개에 비해 훨씬 적은 수, 영상당 98개의 경계 상자만 제안한다. 마지막으로 개별 구성 요소를 단일 모델로 묶어 공동으로 최적화한다.

기타 빠른 검출기. Fast R-CNN과 Faster R-CNN은 연산 공유와 선택적 탐색 대신 영역을 제안하는 신경망을 사용하여 R-CNN 프레임워크의 속도를 높이는 데 초점을 둔다 [14, 28]. R-CNN보다 속도와 정확도가 개선되지만 여전히 실시간에는 못 미친다.

많은 연구가 DPM 파이프라인 가속에 집중한다 [31, 38, 5]. HOG 계산을 빠르게 하고, 캐스케이드를 쓰며, 연산을 GPU로 옮긴다. 그러나 실시간으로 동작하는 것은 30Hz DPM [31]뿐이다.

거대한 검출 파이프라인의 개별 구성 요소를 최적화하려 하기보다, YOLO는 파이프라인 자체를 버리고 설계상 빠르다.

얼굴이나 사람처럼 단일 클래스용 검출기는 훨씬 적은 변동만 다루면 되므로 크게 최적화할 수 있다 [37]. YOLO는 여러 종류의 객체를 동시에 검출하는 범용 검출기로 학습된다.

Deep MultiBox. R-CNN과 달리 Szegedy 등 [8]은 선택적 탐색 대신 관심 영역을 예측하도록 합성곱 신경망을 학습한다. MultiBox는 신뢰도 예측을 단일 클래스 예측으로 바꾸면 단일 객체 검출에도 쓸 수 있다. 그러나 MultiBox는 일반 객체 검출을 수행할 수 없으며 더 큰 파이프라인의 한 조각으로, 추가적인 영상 패치 분류가 필요하다. YOLO와 MultiBox 모두 합성곱 네트워크로 경계 상자를 예측하지만, YOLO는 완전한 검출 시스템이다.

OverFeat. Sermanet 등 [32]은 국소화를 수행하도록 합성곱 신경망을 학습하고 이 국소화기를 검출에 맞춘다 [32]. OverFeat는 슬라이딩 윈도우 검출을 효율적으로 수행하지만 여전히 분리된 시스템이다. OverFeat는 국소화를 최적화할 뿐 검출 성능은 아니다. DPM처럼 국소화기는 예측 시 국소 정보만 본다. OverFeat는 전역 문맥을 추론할 수 없어 일관된 검출을 만들려면 상당한 후처리가 필요하다.

MultiGrasp. 본 연구는 Redmon 등의 파지(grasp) 검출 연구 [27]와 설계가 비슷하다. 경계 상자 예측을 위한 격자 접근은 회귀를 통해 파지를 예측하는 MultiGrasp 시스템에 기반한다. 다만 파지 검출은 객체 검출보다 훨씬 단순한 과제이다. MultiGrasp는 객체 하나가 들어 있는 영상에 대해 파지 가능 영역 하나만 예측하면 된다. 객체의 크기·위치·경계를 추정하거나 클래스를 예측할 필요는 없고, 파지에 적합한 영역만 찾으면 된다. YOLO는 한 영상에서 여러 객체의 경계 상자와 클래스 확률을 동시에 예측한다.

4. 실험

먼저 PASCAL VOC 2007에서 YOLO를 다른 실시간 검출 시스템과 비교한다. YOLO와 R-CNN 변형의 차이를 이해하기 위해 VOC 2007에서 YOLO와 Fast R-CNN(상위 성능 R-CNN 버전 중 하나 [14])이 범하는 오류를 분석한다. 서로 다른 오류 프로필을 바탕으로 YOLO가 Fast R-CNN 검출을 재점수화하여 배경 거짓 양성 오류를 줄이면 성능이 크게 향상됨을 보인다. 또한 VOC 2012 결과를 제시하고 mAP를 최신 방법과 비교한다. 마지막으로 두 예술 작품 데이터셋에서 YOLO가 다른 검출기보다 새로운 영역에 더 잘 일반화함을 보인다.

4.1. 다른 실시간 시스템과의 비교

객체 검출 연구의 많은 노력이 표준 검출 파이프라인을 빠르게 만드는 데 쏠려 있다 [5, 38, 31, 14, 17, 28]. 그러나 Sadeghi 등 [31]이 실제로 실시간(초당 30프레임 이상) 검출 시스템을 만든 경우이다. YOLO를 GPU에서 30Hz 또는 100Hz로 동작하는 DPM 구현과 비교한다. 다른 노력들은 실시간 기준에 미치지 못하지만, 객체 검출에서 이용 가능한 정확도–속도 절충을 살펴보기 위해 상대적 mAP와 속도도 함께 비교한다.

Fast YOLO는 PASCAL에서 가장 빠른 객체 검출 방법이며, 알려진 바로는 가장 빠른 객체 검출기이다. mAP 52.7%로 이전 실시간 검출 연구보다 두 배 이상 정확하다. YOLO는 mAP를 63.4%까지 끌어올리면서도 실시간 성능을 유지한다.

VGG-16을 사용하여 YOLO도 학습한다. 이 모델은 YOLO보다 정확하지만 훨씬 느리다. VGG-16에 의존하는 다른 검출 시스템과 비교하는 데 유용하지만 실시간보다 느리므로 이후 논문은 더 빠른 모델에 초점을 둔다.

Fastest DPM [38]은 mAP를 크게 희생하지 않고 DPM을 가속하지만 실시간에는 여전히 2배 가량 모자라다. 또한 신경망 접근에 비해 DPM의 검출 정확도가 상대적으로 낮다는 점에 제약을 받는다.

R-CNN minus R [20]은 선택적 탐색 대신 정적 경계 상자 제안을 사용한다. R-CNN보다 훨씬 빠르지만 여전히 실시간은 아니며 좋은 제안이 없어 정확도도 크게 떨어진다.

Fast R-CNN은 R-CNN의 분류 단계를 빠르게 하지만 여전히 경계 상자 제안에는 선택적 탐색에 의존하며, 영상당 약 2초가 걸릴 수 있다. 따라서 mAP는 높지만 초당 0.5프레임으로 실시간과는 거리가 멀다.

최근 Faster R-CNN은 Szegedy 등 [8]과 비슷하게 선택적 탐색 대신 신경망으로 경계 상자를 제안한다. 시험에서 가장 정확한 모델은 초당 7프레임이고, 더 작고 덜 정확한 모델은 초당 18프레임이다. VGG-16 버전 Faster R-CNN은 mAP가 10포인트 더 높지만 YOLO보다 6배 느리다. Zeiler–Fergus Faster R-CNN은 YOLO보다 2.5배 느리기만 할 뿐 정확도도 낮다.

표 1: PASCAL VOC 2007에서의 실시간 시스템. 빠른 검출기의 성능과 속도를 비교한다. Fast YOLO는 PASCAL VOC 검출에서 기록상 가장 빠른 검출기이면서 다른 실시간 검출기의 두 배에 해당하는 정확도를 낸다. YOLO는 빠른 버전보다 mAP가 10포인트 높으면서도 여전히 실시간 이상의 속도를 유지한다.

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	**155**
YOLO	2007+2012	**63.4**	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

4.2. VOC 2007 오류 분석

YOLO와 최첨단 검출기의 차이를 더 살펴보기 위해 VOC 2007 결과의 세부 오류 분해를 본다. PASCAL에서 상위 성능 검출기 중 하나인 Fast R-CNN과 YOLO를 비교하며, Fast R-CNN의 검출 결과는 공개되어 있다.

Hoiem 등 [19]의 방법과 도구를 사용한다. 테스트 시 각 범주에 대해 상위 N개 예측을 보며, 각 예측은 정답이거나 오류 유형에 따라 분류된다.

- 정답: 클래스가 맞고 IOU > 0.5
- 국소화: 클래스는 맞고 $0.1 < \text{IOU} < 0.5$
- 유사: 클래스가 비슷하고 IOU > 0.1
- 기타: 클래스가 틀리고 IOU > 0.1
- 배경: 어떤 객체에 대해서도 IOU < 0.1

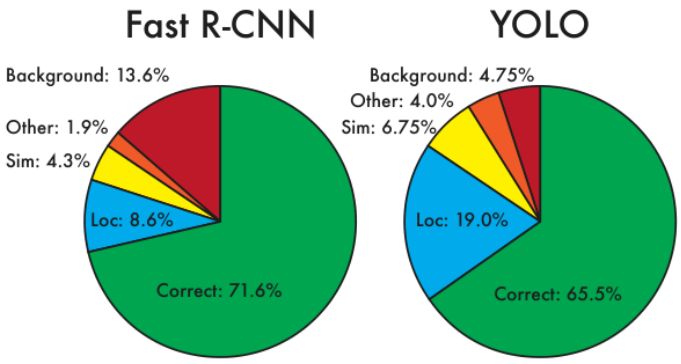


그림 4: 오류 분석: Fast R-CNN과 YOLO. 여러 범주에 대해 상위 N개 검출(N은 해당 범주의 객체 수)에서 국소화 오류와 배경 오류 비율을 나타낸다.

그림 4는 20개 클래스 전체에 평균한 각 오류 유형의 비율을 나타낸다.

YOLO는 객체를 올바르게 국소화하는 데 어려움을 겪는다. 국소화 오류는 YOLO 오류의 다른 모든 원인을 합친 것보다 크다. Fast R-CNN은 국소화 오류는 훨씬 적지만 배경 오류는 훨씬 많다. 상위 검출의 13.6%는 어떤 객체도 포함하지 않는 거짓 양성이다. Fast R-CNN은 YOLO보다 배경 검출을 예측할 가능성이 거의 3배 크다.

4.3. Fast R-CNN과 YOLO의 결합

YOLO는 Fast R-CNN보다 배경 실수를 훨씬 적게 낸다. YOLO를 사용하여 Fast R-CNN의 배경 검출을 제거하면 성능이 크게 오른다. R-CNN이 예측한 각 경계 상자에 대해 YOLO가 비슷한 상자를 예측하는지 확인하고, 그럴 경우 YOLO가 예측한 확률과 두 상자의 겹침에 따라 해당 예측에 가중을 준다.

최고의 Fast R-CNN 모델은 VOC 2007 테스트 집합에서 mAP 71.8%를 달성한다. YOLO와 결합하면 mAP가 3.2%포인트 올라 75.0%가 된다. 최상위 Fast R-CNN 모델과 Fast R-CNN의 여러 다른 버전을 결합해 보기도 했으며, 그 앙상블은 mAP를 0.3–0.6%포인트 정도만 올렸고, 자세한 내용은 표 2를 보라.

YOLO로 인한 향상이 단순히 모델 앙상블의 부산물은 아니다. 서로 다른 Fast R-CNN 버전을 결합해도 이득이 거의 없기 때문이다. 오히려 테스트 시 YOLO가 다른 종류의 실수를 하기 때문에 Fast R-CNN의 성능을 끌어올리는 데 효과적이다.

안타깝게도 이 결합은 YOLO의 속도 이점을 살리지 못한다. 각 모델을 따로 실행한 뒤 결과를 합치기 때문이다. 다만 YOLO가 매우 빨라 Fast R-CNN에 비해 유의미한 계산 시간을 더하지는 않는다.

표 2: VOC 2007에서의 모델 결합 실험. 최고 성능 Fast R-CNN과 여러 모델을 결합하는 효과를 검토한다. 다른 버전의 Fast R-CNN은 소폭의 이득만 주는 반면 YOLO는 성능을 크게 높인다.

	mAP	Combined	Gain
Fast R-CNN	71.8	-	-
Fast R-CNN (2007 data)	**66.9**	72.4	.6
Fast R-CNN (VGG-M)	59.2	72.4	.6
Fast R-CNN (CaffeNet)	57.1	72.1	.3
YOLO	63.4	**75.0**	**3.2**

표 3: PASCAL VOC 2012 리더보드. 2015년 11월 6일 기준 전체 comp4(외부 데이터 허용) 공개 리더보드에서 YOLO와의 비교이다. 다양한 검출 방법에 대해 평균 정밀도와 범주별 평균 정밀도를 나타낸다. YOLO만이 실시간 검출기이다. Fast R-CNN + YOLO는 Fast R-CNN 대비 2.3%포인트 향상으로 네 번째로 높은 점수를 기록한 방법이다.

VOC 2012 test	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
MR_CNN_MORE_DATA [11]	**73.9**	**85.5**	**82.9**	**76.6**	**57.8**	**62.7**	**79.4**	77.2	86.6	**55.0**	**79.1**	**62.2**	87.0	**83.4**	**84.7**	78.9	45.3	73.4	65.8	80.3	74.0
HyperNet_VGG	71.4	84.2	78.5	73.6	55.6	53.7	78.7	**79.8**	87.7	49.6	74.9	52.1	86.0	81.7	83.3	**81.8**	**48.6**	**73.5**	59.4	79.9	65.7
HyperNet_SP	71.3	84.1	78.3	73.3	55.5	53.6	78.6	79.6	87.5	49.5	74.9	52.1	85.6	81.6	83.2	81.6	48.4	73.2	59.3	79.7	65.6
Fast R-CNN + YOLO	70.7	83.4	78.5	73.5	55.8	43.4	79.1	73.1	**89.4**	49.4	75.5	57.0	**87.5**	80.9	81.0	74.7	41.8	71.5	68.5	**82.1**	67.2
MR_CNN_S_CNN [11]	70.7	85.0	79.6	71.5	55.3	57.7	76.0	73.9	84.6	50.5	74.3	61.7	85.5	79.9	81.7	76.4	41.0	69.0	61.2	77.7	72.1
Faster R-CNN [28]	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
DEEP_ENS_COCO	70.1	84.0	79.4	71.6	51.9	51.1	74.1	72.1	88.6	48.3	73.4	57.8	86.1	80.0	80.7	70.4	46.6	69.6	**68.8**	75.9	71.4
NoC [29]	68.8	82.8	79.0	71.6	52.3	53.7	74.1	69.0	84.9	46.9	74.3	53.1	85.0	81.3	79.5	72.2	38.9	72.4	59.5	76.7	68.1
Fast R-CNN [14]	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	**87.5**	80.5	80.8	72.0	35.1	68.3	65.7	80.4	64.2
UMICH_FGS_STRUCT	66.4	82.9	76.1	64.1	44.6	49.4	70.3	71.2	84.6	42.7	68.6	55.8	82.7	77.1	79.9	68.7	41.4	69.0	60.0	72.0	66.2
NUS_NIN_C2000 [7]	63.8	80.2	73.8	61.9	43.7	43.0	70.3	67.6	80.7	41.9	69.7	51.7	78.2	75.2	76.9	65.1	38.6	68.3	58.0	68.7	63.3
BabyLearning [7]	63.2	78.0	74.2	61.3	45.7	42.7	68.2	66.8	80.2	40.6	70.0	49.8	79.0	74.5	77.9	64.0	35.3	67.9	55.7	68.7	62.6
NUS_NIN	62.4	77.9	73.1	62.6	39.5	43.3	69.1	66.4	78.9	39.1	68.1	50.0	77.2	71.3	76.1	64.7	38.4	66.9	56.2	66.9	62.7
R-CNN VGG BB [13]	62.4	79.6	72.7	61.9	41.2	41.9	65.9	66.4	84.6	38.5	67.2	46.7	82.0	74.8	76.0	65.2	35.6	65.4	54.2	67.4	60.3
R-CNN VGG [13]	59.2	76.8	70.9	56.6	37.5	36.9	62.9	63.6	81.1	35.7	64.3	43.9	80.4	71.6	74.0	60.0	30.8	63.4	52.0	63.5	58.7
YOLO	57.9	77.0	67.2	57.7	38.3	22.7	68.3	55.9	81.4	36.2	60.8	48.5	77.2	72.3	71.3	63.5	28.9	52.2	54.8	73.9	50.8
Feature Edit [33]	56.3	74.6	69.1	54.4	39.1	33.1	65.2	62.7	69.7	30.8	56.0	44.6	70.0	64.4	71.1	60.2	33.3	61.3	46.4	61.7	57.8
R-CNN BB [13]	53.3	71.8	65.8	52.0	34.1	32.6	59.6	60.0	69.8	27.6	52.0	41.7	69.6	61.3	68.3	57.8	29.6	57.8	40.9	59.3	54.1
SDS [16]	50.7	69.7	58.4	48.5	28.3	28.8	61.3	57.5	70.8	24.1	50.7	35.9	64.9	59.1	65.8	57.1	26.0	58.8	38.6	58.9	50.7
R-CNN [13]	49.6	68.1	63.8	46.1	29.4	27.9	56.6	57.0	65.9	26.5	48.7	39.5	66.2	57.3	65.4	53.2	26.2	54.5	38.1	50.6	51.6

4.4. VOC 2012 결과

VOC 2012 테스트 집합에서 YOLO는 mAP 57.9%를 기록한다. 이는 최첨단보다 낮고 VGG-16을 쓴 초기 R-CNN에 가깝다(표 3). 본 시스템은 가장 가까운 경쟁자들에 비해 작은 객체에 어려움이 있다. bottle, sheep, tv/monitor 같은 범주에서는 R-CNN이나 Feature Edit보다 8–10%포인트 낮다. 그러나 cat, train 등 다른 범주에서는 YOLO가 더 높은 성능을 낸다.

결합된 Fast R-CNN + YOLO 모델은 상위 성능 검출 방법 중 하나이다. Fast R-CNN은 YOLO와 결합해 2.3%포인트 개선되며 공개 리더보드에서 5계단 상승한다.

4.5. 일반화: 예술 작품에서의 사람 검출

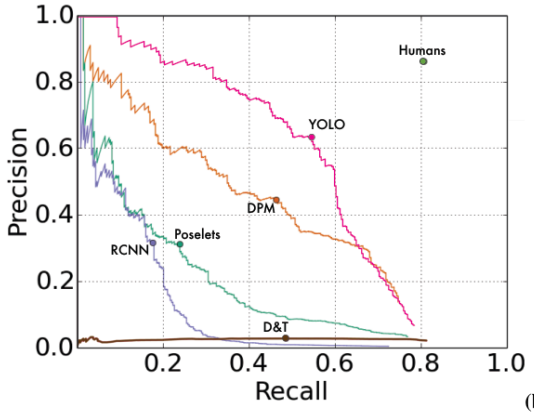
객체 검출용 학술 데이터셋은 학습과 테스트 데이터를 같은 분포에서 가져온다. 실제 응용에서는 가능한 모든 사용 사례를 예측하기 어렵고 테스트 데이터가 시스템이 본 것과 달라질 수 있다 [3]. Picasso 데이터셋 [12]과 People-Art 데이터셋 [3]에서 예술 작품에 대한 사람 검출을 시험하며 YOLO를 다른 검출 시스템과 비교한다.

그림 5는 YOLO와 다른 검출 방법의 비교 성능을 나타낸다. 참고로 모든 모델을 VOC 2007 데이터만으로 학습했을 때 사람에 대한 VOC 2007 검출 AP를 함께 제시한다. Picasso에서는 모델을 VOC 2012로, People-Art에서는 VOC 2010으로 학습했다.

R-CNN은 VOC 2007에서 AP가 높지만 예술 작품에 적용하면 크게 떨어진다. R-CNN은 자연 영상에 맞춰진 선택적 탐색으로 경계 상자 제안을 얻는다. R-CNN의 분류 단계는 작은 영역만 보고 좋은 제안이 필요하다.

DPM은 예술 작품에 적용해도 AP가 잘 유지된다. 선행 연구는 DPM이 객체의 형태와 배치에 대한 강한 공간 모델을 갖기 때문에 잘 동작한다고 설명한다. DPM이 R-CNN만큼 크게 떨어지지는 않지만 초기 AP는 더 낮다.

YOLO는 VOC 2007에서 좋은 성능을 내고, 예술 작품에 적용할 때 다른 방법보다 AP가 덜 감소한다. DPM처럼 YOLO도 객체의 크기와 형태, 객체 간 관계, 객체가 자주 나타나는 위치를 모델링한다. 예술 작품과 자연 영상은 픽셀 수준에서는 매우 다르지만 객체의 크기와 형태는 비슷하므로 YOLO는 여전히 좋은 경계 상자와 검출을 예측할 수 있다.



(a) Picasso Dataset precision-recall curves.

	VOC 2007	Picasso		People-Art
	AP	AP	Best F_1	AP
YOLO	**59.2**	**53.3**	**0.590**	**45**
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

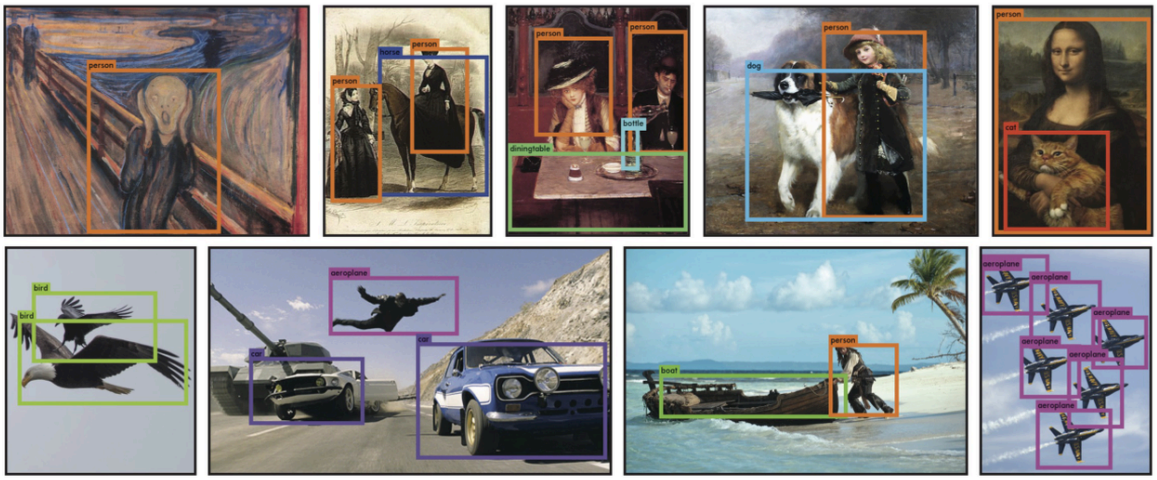


그림 5: (위) Picasso·People-Art 데이터셋에 대한 일반화 결과(정밀도–재현율 곡선 및 표). (아래) 정성적 결과: 인터넷의 예술 작품과 자연 영상 샘플에서 실행한 YOLO. 대체로 정확하지만 한 사람을 비행기로 잘못 본 경우도 있다.

5. 실제 환경에서의 실시간 검출

YOLO는 빠르고 정확한 객체 검출기이므로 컴퓨터 비전 응용에 적합하다. 웹캠에 연결하여 실시간 성능이 유지되는지 확인하였으며, 여기에는 카메라에서 영상을 가져오고 검출을 화면에 그리는 시간도 포함된다.

결과 시스템은 대화형이고 사용 경험이 좋다. YOLO는 영상을 각각 처리하지만 웹캠에 붙이면 추적 시스템처럼 동작하며 객체가 움직이고 겉모습이 바뀌는 동안 검출한다. 시연과 소스 코드는 프로젝트 웹사이트 http://pjreddie.com/yolo/에서 볼 수 있다.

6. 결론

본 논문은 객체 검출을 위한 통합 모델인 YOLO를 소개한다. 모델은 구성이 단순하고 전체 영상에 직접 학습할 수 있다. 분류기 기반 접근과 달리 YOLO는 검출 성능에 직접 대응하는 손실 함수로 학습하며 전체 모델을 공동으로 학습한다.

Fast YOLO는 문헌에서 가장 빠른 범용 객체 검출기이고, YOLO는 실시간 객체 검출에서 최첨단을 밀어 올린다. YOLO는 새로운 영역에도 잘 일반화되어 빠르고 견고한 객체 검출에 의존하는 응용에 적합하다.

사사. 본 연구는 ONR N00014-13-1-0720, NSF IIS-1338054, Allen Distinguished Investigator Award의 지원을 일부 받았다.

References

[1] M. B. Blaschko and C. H. Lampert. Learning to localize objects with structured output regression. In Computer Vision–ECCV 2008, pages 2–15. Springer, 2008.

[2] L. Bourdev and J. Malik. Poselets: Body part detectors trained using 3d human pose annotations. In International Conference on Computer Vision (ICCV), 2009.

[3] H. Cai, Q. Wu, T. Corradi, and P. Hall. The cross-depiction problem: Computer vision algorithms for recognising objects in artwork and in photographs. arXiv preprint arXiv:1505. 00110, 2015.

[4] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In Computer Vision and Pattern Recognition, 2005. CVPR 2005. IEEE Computer Society Conference on, volume 1, pages 886–893. IEEE, 2005.

[5] T. Dean, M. Ruzon, M. Segal, J. Shlens, S. Vijayanarasimhan, J. Yagnik, et al. Fast, accurate detection of 100,000 object classes on a single machine. In Computer Vision and Pattern Recognition (CVPR), 2013 IEEE Conference on, pages 1814–1821. IEEE, 2013.

[6] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell. Decaf: A deep convolutional activation feature for generic visual recognition. arXiv preprint arXiv:1310.1531, 2013.

[7] J. Dong, Q. Chen, S. Yan, and A. Yuille. Towards unified object detection and semantic segmentation. In Computer Vision–ECCV 2014, pages 299–314. Springer, 2014.

[8] D. Erhan, C. Szegedy, A. Toshev, and D. Anguelov. Scalable object detection using deep neural networks. In Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on, pages 2155–2162. IEEE, 2014.

[9] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The pascal visual object classes challenge: A retrospective. International Journal of Computer Vision, 111(1):98–136, Jan. 2015.

[10] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, and D. Ramanan. Object detection with discriminatively trained part based models. IEEE Transactions on Pattern Analysis and Machine Intelligence, 32(9):1627–1645, 2010.

[11] S. Gidaris and N. Komodakis. Object detection via a multi-region & semantic segmentation-aware CNN model. CoRR, abs/1505.01749, 2015.

[12] S. Ginosar, D. Haas, T. Brown, and J. Malik. Detecting people in cubist art. In Computer Vision -ECCV 2014 Workshops, pages 101–116. Springer, 2014.

[13] R. Girshick, J. Donahue, T. Darrell, and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on, pages 580–587. IEEE, 2014.

[14] R. B. Girshick. Fast R-CNN. CoRR, abs/1504.08083, 2015.

[15] S. Gould, T. Gao, and D. Koller. Region-based segmentation and object detection. In Advances in neural information processing systems, pages 655–663, 2009.

[16] B. Hariharan, P. Arbeláez, R. Girshick, and J. Malik. Simultaneous detection and segmentation. In Computer Vision–ECCV 2014, pages 297–312. Springer, 2014.

[17] K. He, X. Zhang, S. Ren, and J. Sun. Spatial pyramid pooling in deep convolutional networks for visual recognition. arXiv preprint arXiv:1406.4729, 2014.

[18] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. arXiv preprint arXiv:1207.0580, 2012.

[19] D. Hoiem, Y. Chodpathumwan, and Q. Dai. Diagnosing error in object detectors. In Computer Vision–ECCV 2012, pages 340–353. Springer, 2012.

[20] K. Lenc and A. Vedaldi. R-cnn minus r. arXiv preprint arXiv:1506.06981, 2015.

[21] R. Lienhart and J. Maydt. An extended set of haar-like features for rapid object detection. In Image Processing. 2002. Proceedings. 2002 International Conference on, volume 1, pages I–900. IEEE, 2002.

[22] M. Lin, Q. Chen, and S. Yan. Network in network. CoRR, abs/1312.4400, 2013.

[23] D. G. Lowe. Object recognition from local scale-invariant features. In computer vision, 1999. The proceedings of the seventh IEEE international conference on, volume 2, pages 1150–1157. Ieee, 1999.

[24] D. Mishkin. Models accuracy on imagenet 2012 val. https://github.com/BVLC/caffe/wiki/Models-accuracy-on-ImageNet-2012-val. Accessed: 2015-10-2.

[25] C. P. Papageorgiou, M. Oren, and T. Poggio. A general framework for object detection. In Computer vision, 1998. sixth international conference on, pages 555–562. IEEE, 1998.

[26] J. Redmon. Darknet: Open source neural networks in c. http://pjreddie.com/darknet/, 2013–2016.

[27] J. Redmon and A. Angelova. Real-time grasp detection using convolutional neural networks. CoRR, abs/1412.3128, 2014.

[28] S. Ren, K. He, R. Girshick, and J. Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. arXiv preprint arXiv:1506.01497, 2015.

[29] S. Ren, K. He, R. B. Girshick, X. Zhang, and J. Sun. Object detection networks on convolutional feature maps. CoRR, abs/1504.06066, 2015.

[30] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. International Journal of Computer Vision (IJCV), 2015.

[31] M. A. Sadeghi and D. Forsyth. 30hz object detection with dpm v5. In Computer Vision–ECCV 2014, pages 65–79. Springer, 2014.

[32] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. CoRR, abs/1312.6229, 2013.

[33] Z. Shen and X. Xue. Do more dropouts in pool5 feature maps for better object detection. arXiv preprint arXiv:1409.6911, 2014.

[34] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. CoRR, abs/1409.4842, 2014.

[35] J. R. Uijlings, K. E. van de Sande, T. Gevers, and A. W. Smeulders. Selective search for object recognition. International journal of computer vision, 104(2):154–171, 2013.

[36] P. Viola and M. Jones. Robust real-time object detection. International Journal of Computer Vision, 4:34–47, 2001.

[37] P. Viola and M. J. Jones. Robust real-time face detection. International journal of computer vision, 57(2):137–154, 2004.

[38] J. Yan, Z. Lei, L. Wen, and S. Z. Li. The fastest deformable part model for object detection. In Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on, pages 2497–2504. IEEE, 2014.

[39] C. L. Zitnick and P. Dollár. Edge boxes: Locating object proposals from edges. In Computer Vision–ECCV 2014, pages 391–405. Springer, 2014.